MICROSERVICES COURSE · PHASE 2 · SESSION 9 OF 29 · ANCHOR DAY

Docker Containerization

Multi-Stage Builds · Docker Compose · Health Checks · Production Images

Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

DAY	TIME	DURATION
Wednesday · In-Person	**10:00 AM – 3:00 PM + Exam**	**5 Hours · Offline Anchor Day**

WHERE WE ARE

Phase 2 — Quality & Deployment

Phase 1	● LIVE Phase 2	Phase 3	Phase 4

S9	S10	S11	S12	S13	S14	S15	S16
Docker	Testing I	Testing II	Saga Orch.	CI/CD	GitOps	K8s Core	K8s Adv.
● LIVE							

Today: Phase 1 code meets Phase 2 reliability — same image, every environment, every time.

Full 29-session roadmap: see Course Kickoff deck

PHASE 2 BEGINS

From Writing Code to Shipping It Reliably

Phase 1 — Sessions 1–8

Wrote the services.
Built the patterns.
Proved the architecture works on our laptops.

Phase 2 — Sessions 9–16

Containerize it.
Test it rigorously.
Deploy it to Kubernetes.
Ship it with CI/CD.
Make it production-worthy.

THE PROBLEM

“It Works on My Machine”

The Story

- A trainee sends Order Service to a colleague — colleague gets: Cannot connect to Kafka on localhost:9092
- Original dev runs Kafka locally. Colleague does not. Different OS, different JDK patch, different env vars.
- CI server uses JDK 17 instead of 21 — build passes but runtime crashes.
- Result: 2 hours debugging environment differences instead of business logic.

WHY IT FAILS

Three Root Causes of Environment Inconsistency

JDK Version Mismatch

Developer uses JDK 21.
CI uses JDK 17.
Production uses 21 — but a different patch level with a different GC default.
The JAR compiles fine but crashes at runtime due to API differences.

Missing Environment Variables

SPRING_PROFILES_ACTIVE not set in CI.
SPRING_DATASOURCE_URL points to localhost (valid on laptop, unreachable in CI).
No env var means Spring falls back to a wrong default.

localhost vs Container Hostname

localhost inside a container refers to the container itself — not the host machine, not PostgreSQL, not Kafka.
A service configured for localhost fails the moment it moves off the developer laptop.

THE SOLUTION

What Containers Solve: Three Principles

Isolation

Each service runs in its own process space, filesystem, and network namespace.

*Order Service on port 8082 **cannot** accidentally bind to Product's 8081 — even if both run on the same host.*

Reproducibility

Build once, run anywhere — the image captures the exact runtime environment.

*CI server, staging, and developer laptops all run the **same image. Same JDK, same dependencies, same behavior**.*

Portability

An image runs on any host with Docker — OS-independent.

*Develop on Windows, deploy to Linux in production. **No changes required between environments**.*

We've run Docker Compose since Session 1 for PostgreSQL, Kafka, Redis. Today we containerize OUR services — the Spring Boot apps we wrote.

Image vs Container — Two Different Things

Docker Image

A frozen, read-only snapshot.

- Built from a **Dockerfile** — deterministic, versioned
- Stored in a registry (Docker Hub, ECR, GHCR)
- **Immutable** — running it doesn't change it
- Think: a **class** definition in Java, or a ZIP file

Docker Container

A running instance of an image.

- Writable layer added on top of the image at runtime
- **Isolated process** with its own network, filesystem
- Ephemeral by default — dies without a volume
- Think: an **object** instance created from the class

Every Instruction = One Layer — Cache Reuse Is the Key

`FROM eclipse-temurin:21`	Base layer — pulled once, cached forever
`COPY pom.xml .`	pom.xml layer — changes rarely → stable
`RUN mvn dependency:go-offline`	Dependencies layer — 2 min download, cached if pom unchanged
`COPY src ./src`	Source layer — changes every commit
`RUN mvn package`	Build layer — re-runs only when src changes

Rule: put stable layers ABOVE volatile layers. If pom.xml hasn't changed, Maven download is skipped — saves ~2 minutes on every rebuild.

THE NAIVE APPROACH

Single-Stage Build: Everything Goes into the Image

```
FROM eclipse-temurin:21
WORKDIR /app

# Install Maven INSIDE the image
RUN apt-get install -y maven

# Copy everything
COPY . .

# Build
RUN mvn package -DskipTests

CMD ["java", "-jar", "target/product-service.jar"]
```

WHAT'S INSIDE THIS IMAGE

- JDK 21 — full compiler, jshell, all tools
- Maven + local .m2 repository (~300 MB)
- All source code (ProductService.java etc.)
- Test classes and test dependencies
- Build metadata and intermediate files

~700 MB

Build tools in production = security risk + slow deploys

THE SOLUTION

Multi-Stage Build: Single-Stage vs Multi-Stage

SINGLE-STAGE (naive)

```
FROM eclipse-temurin:21
WORKDIR /app

# Install Maven INSIDE the image
RUN apt-get install -y maven

# Copy everything
COPY . .

# Build
RUN mvn package -DskipTests

CMD ["java", "-jar", "target/product-service.jar"]
```

IMAGE SIZE: ~700 MB
app.jar (includes: JDK + Maven + system --group app source code + test classes)

MULTI-STAGE (production)

```
FROM eclipse-temurin:21 AS builder
WORKDIR /build
COPY pom.xml .
COPY src ./src
RUN mvn package -DskipTests


FROM eclipse-temurin:21-jre-jammy
WORKDIR /app
COPY --from=builder /build/target/*.jar

RUN addgroup --system app && adduser -

USER app
CMD ["java","-jar","app.jar"]
```

IMAGE SIZE: ~240 MB — 60% smaller
(includes: JRE only + final JAR)

THE SOLUTION

Multi-Stage Build: Builder vs Runtime

STAGE 1 — BUILDER

```
FROM eclipse-temurin:21 AS builder
```

- Full JDK — compiler, javac, jshell
- Maven installed, runs mvn package
- Source code copied in
- Result: compiled JAR in /build/target/
- ⚠ This stage is DISCARDED after build

COPY --from=builder

STAGE 2 — RUNTIME

```
FROM eclipse-temurin:21-jre-jammy
```

- JRE only — no compiler, no jshell
- COPY --from=builder *.jar — just the JAR
- Non-root user created and switched to
- EXPOSE + ENTRYPOINT set
- ✓ This stage becomes the final image

Single-stage: ~700 MB

Multi-stage: ~240 MB — 60% smaller

BASE IMAGE SELECTION

eclipse-temurin:21 vs eclipse-temurin:21-jre-jammy

ENGINEERING DECISION

Image	Size	Contains	Use When
`eclipse-temurin:21`	**~700 MB**	JDK + javac + jshell + tools	**Builder stage ONLY**
`eclipse-temurin:21-jre-jammy`	**~240 MB**	JRE only — runs JARs, no compiler	**Runtime stage (production)**

The Rule

Runtime images always get the JRE tag (‡21-jre-jammy). The builder stage gets the full JDK. Never put a full JDK in a production image.

WHY JAMMY?

- Ubuntu 22.04 LTS (‘Jammy Jellyfish’) base — long-term support, security patches guaranteed until 2027
- Smaller attack surface than the non-slim Debian base; well-tested with Spring Boot. Good default for all Phase 2 services.

Stage 1: Builder — pom.xml First for Layer Cache

Dockerfile — product-service — Stage 1: Build

```
# ── Stage 1: Build ──────────────────────────────────────────────
FROM eclipse-temurin:21 AS builder
WORKDIR /build

# Copy dependency descriptor pom.xml FIRST — Docker caches this layer
# if pom.xml unchanged, Docker skips the next layer (Maven download) entirely on rebuild. Saves ~2 minutes.
COPY pom.xml .
RUN mvn dependency:go-offline -B # pre-fetch all deps into layer cache



# Source changes more often than pom.xml — put it AFTER deps
COPY src ./src

RUN mvn package -DskipTests -B # compile + package, skip tests (CI runs them)
```

Stage 2: Runtime — JRE Only, Non-Root User

Dockerfile — product-service — Stage 2: Runtime

```
# ── Stage 2: Runtime ──────────────────────────────────────
FROM eclipse-temurin:21-jre-jammy
WORKDIR /app

# Security: never run as root in production
RUN addgroup --system spring && adduser --system --group spring
USER spring

# Copy ONLY the compiled JAR from the builder stage
COPY --from=builder /build/target/*.jar app.jar

EXPOSE 8081

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Dockerfiles for Platform Services

- One Dockerfile pattern covers all 7 core services.
- Type it once for product-service — copy the pattern for the others.

BUILD CONTEXT HYGIENE

.dockerignore: Keep the Build Context Small

.dockerignore — shared across all services

```
# .dockerignore
# (place in each service module root)

target/
.idea/
.git/
*.md
*.log
**/*.class
```

Exclude build artifacts and IDE files from Docker build context

WHY THIS MATTERS

- Without **.dockerignore**, docker build sends the ENTIRE directory to the daemon — including target/ (200–300 MB of compiled classes)
- Build context is transferred before any layer is processed — a large context slows every single rebuild
- **target/** contains the OLD JAR from your last local build, which Docker might copy instead of the freshly compiled one
- **.git/** leaks commit history into the image (security concern)

docker build — 240 MB vs 700 MB

```
# Build the product-service image
$ cd services/product-service
$ docker build -t product-service:local .
  [+] Building 127.3s (12/12) FINISHED
  => CACHED [builder 3/5] RUN mvn dependency:go-offline    # ✔ cache hit!
  => [builder 4/5] COPY src ./src                          # only src changed
  => [builder 5/5] RUN mvn package -DskipTests
  => [stage-2 3/4] COPY --from=builder *.jar app.jar       # 42 MB JAR only

# Check image size — should be ~240MB NOT ~700MB
$ docker images product-service:local
  REPOSITORY          TAG     IMAGE ID      SIZE
  product-service     local   a3f2b1c9d8e7  241MB
```

Next rebuild (only src changed):

- pom.xml layer: CACHED ✔ (skips 2-min Maven download)
- dependency layer: CACHED ✔
- src layer: runs (changed)
- Rebuild time: ~15s vs ~2min first time

Why the size matters in production:

- Smaller image = faster pull on Kubernetes nodes
- Less attack surface — no compiler vulnerabilities
- Faster CI pipeline: build, push, deploy

docker run — Verify the image

```
# Run standalone to verify it starts
docker run --rm -p 8081:8081 \
  -e SPRING_PROFILES_ACTIVE=docker \
  -e SPRING_CONFIG_IMPORT=optional:configserver:http://host.docker.internal:8888 \
  -e EUREKA_CLIENT_SERVICE_URL_DEFAULTZONE=http://host.docker.internal:8761/eureka/ \
  product-service:local
```

- **host.docker.internal** resolves to the **host machine** from inside a container.
- This is Docker Desktop behavior (Windows/macOS).
- On Linux, use **--add-host=host.docker.internal:host-gateway** instead.

How Services Find Each Other: Service Name = Hostname

platform-net (Docker bridge network)

postgres	eureka-server	order-service	api-gateway
:5432	:8761	:8082	:8080

`order-service → SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/orderdb`

Docker DNS resolves 'postgres' to the container's internal IP on platform-net automatically.

`api-gateway → EUREKA_CLIENT_SERVICEURL_DEFAULTZONE=http://eureka-server:8761/eureka/`

Never use localhost in docker-compose.yml environment variables. localhost inside a container = the container itself, not the host or another service.

STARTUP DEPENDENCIES

depends_on: service_healthy vs service_started

DESIGN CHOICE

If: Spring Boot service with /actuator/health

`Then use: service_healthy`

Guarantees the service is actually ready to accept requests, not just running. Config Server, Eureka, product-service, order-service.

If: PostgreSQL or Redis (non-Spring infra)

`Then use: service_healthy`

Both support HEALTHCHECK: pg_isready (Postgres) and redis-cli ping (Redis). Use service_healthy — they're well-defined.

If: Kafka OSS image (no Actuator endpoint)

`Then use: service_started`

Kafka's OSS image has no HTTP health endpoint. Use service_started + Spring Boot's built-in reconnect retry (spring.kafka.producer.retries).

Rule: prefer service_healthy whenever the image supports it. service_started only when the image has no HTTP health endpoint.

LIVE CODING · DOCKER-COMPOSE.YML

Config Server → Eureka → Product Service

docker-compose.yml — add under 'services:' after existing infrastructure

```
  config-server:
    build: ./platform/config-server
    image: config-server:local
    ports: ["8888:8888"]
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8888/actuator/health"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks: [platform-net]

  eureka-server:
    build: ./platform/eureka-server
    image: eureka-server:local
    ports: ["8761:8761"]
    depends_on:
      config-server:
        condition: service_healthy
    environment:
      - SPRING_CONFIG_IMPORT=optional:configserver:http://config-server:8888
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8761/actuator/health"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks: [platform-net]
```

Product Service

docker-compose.yml — Product-service

```
  product-service:
    build: ./services/product-service
    image: product-service:local
    ports: ["8081:8081"]
    depends_on:
      eureka-server:
        condition: service_healthy
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    environment:
      - SPRING_CONFIG_IMPORT=optional:configserver:http://config-server:8888
      - EUREKA_CLIENT_SERVICEURL_DEFAULTZONE=http://eureka-server:8761/eureka/
      - SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/productdb
      - SPRING_DATA_REDIS_HOST=redis
    networks: [platform-net]

    # DEV ONLY: remove healthcheck dependency on redis for faster iteration
```

Order · Payment

docker-compose.yml — order-service, payment-service

```
  order-service:
    build: ./services/order-service
    image: order-service:local
    ports: ["8082:8082"]
    depends_on:
      eureka-server:
        condition: service_healthy
      postgres:
        condition: service_healthy
      kafka:
        condition: service_started   # Kafka has no Spring Actuator healthcheck
    environment:
      - SPRING_CONFIG_IMPORT=optional:configserver:http://config-server:8888
      - EUREKA_CLIENT_SERVICEURL_DEFAULTZONE=http://eureka-server:8761/eureka/
      - SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/orderdb
      - SPRING_KAFKA_BOOTSTRAPSERVERS=kafka:9092
    networks: [platform-net]

  payment-service:
    build: ./services/payment-service
    image: payment-service:local
    ports: ["8083:8083"]
    depends_on:
      eureka-server:
        condition: service_healthy
    environment:
      - SPRING_CONFIG_IMPORT=optional:configserver:http://config-server:8888
      - EUREKA_CLIENT_SERVICEURL_DEFAULTZONE=http://eureka-server:8761/eureka/
      - PAYMENT_FAILURE_RATE=0.0   # DEV ONLY: set to 0 in container for stable demo
    networks: [platform-net]
```

Inventory · API Gateway

docker-compose.yml — inventory-service, api-gateway

```
  inventory-service:
    build: ./services/inventory-service
    image: inventory-service:local
    ports: ["8084:8084"]
    depends_on:
      eureka-server:
        condition: service_healthy
      postgres:
        condition: service_healthy
    environment:
      - SPRING_CONFIG_IMPORT=optional:configserver:http://config-server:8888
      - EUREKA_CLIENT_SERVICEURL_DEFAULTZONE=http://eureka-server:8761/eureka/
      - SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/inventorydb
    networks: [platform-net]

  api-gateway:
    build: ./platform/api-gateway
    image: api-gateway:local
    ports: ["8080:8080"]
  depends_on:
      eureka-server:
        condition: service_healthy
      redis:
        condition: service_healthy
    environment:
      - SPRING_CONFIG_IMPORT=optional:configserver:http://config-server:8888
      - EUREKA_CLIENT_SERVICEURL_DEFAULTZONE=http://eureka-server:8761/eureka/
      - SPRING_DATA_REDIS_HOST=redis
    networks: [platform-net]
```

Networks

docker-compose.yml — networks

```
networks:
  platform-net:
    driver: bridge
    # All services on same network — use service name as hostname
    # e.g. http://eureka-server:8761 resolves inside containers
```

CONFIGURATION STRATEGY

Env Var Injection vs application.yml — When Containers Change the Rules

DESIGN CHOICE

If: Value changes between environments (dev / staging / prod)

`Then: Environment variable in docker-compose.yml`

SPRING_DATASOURCE_URL, EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, REDIS_HOST.
Same image, different env vars = different config.
Never hardcode these in application.yml.

If: Value is the same in every environment (server.port, logging.level)

`Then: application.yml`

Port, log format, feature flags that never change between environments.
Baking these into application.yml is correct — they're not secrets and don't vary.

If: Secret value (password, API key, token)

`Then: Never in docker-compose.yml plaintext — use Docker Secrets or Vault (Phase 3)`

docker-compose.yml is committed to git.
Secrets in env vars = secrets in your repo history.
This is addressed properly in Session 19 (Security + Keycloak).

Verify End-to-End

```
# 1. Build all images
docker compose build

# 2. Start all services
docker compose up -d

# 3. Watch health checks stabilize (takes ~60s)
docker compose ps
# Expected: all services 'healthy' (not 'starting')

# 4. Test the platform end-to-end via Gateway
curl http://localhost:8080/api/products
# Expected: JSON array of products (served from containerized Product Service)

# 5. Check Eureka — all services registered
open http://localhost:8761
# Expected: 7 services visible (config/eureka/gateway/product/order/payment/inventory)

# 6. Verify image sizes
docker images | grep ':local'
# Expected: each image ~240-280MB (not ~700MB)
```

docker compose up -d — Platform Starts in Containers

```
$ docker compose up -d
  [+] Running 12/12
  ✓ Container postgres          Started    (healthy)
  ✓ Container redis             Started    (healthy)
  ✓ Container kafka             Started    (started)    # service_started
  ✓ Container config-server     Started    (healthy)
  ✓ Container eureka-server     Started    (healthy)
  ✓ Container product-service   Started    (healthy)
  ✓ Container order-service     Started    (healthy)
  ✓ Container payment-service   Started    (healthy)
  ✓ Container inventory-service Started    (healthy)
  ✓ Container api-gateway       Started    (healthy)

$ curl http://localhost:8080/api/products
  [{"id":1,"name":"Laptop","price":999.99},{"id":2,"name":"Mouse","price":29.99}]
```

Startup chain: postgres/redis → config-server (healthy) → eureka-server (healthy) → all app services (healthy). Total: ~60–90 seconds on first run.

LAB 8A

Dockerize the Platform

60 minutes (12:45–13:45) · Scope: all 7 core Phase 1 services · Grade: Feature 70% + Tests 20% + Quality 10%

1 Write Dockerfiles

20 min

- Copy the product-service multi-stage Dockerfile pattern to ALL 7 core services.
- Change only EXPOSE port per service.
- Add .dockerignore to every module root.

2 Update docker-compose.yml

20 min

- Add all 7 services.
- Each needs: build path, ports, depends_on (service_healthy / service_started for Kafka), environment vars, networks: [platform-net].

3 Verify End-to-End

15 min

- docker compose build → up -d → ps (all healthy).
- curl /api/products via Gateway.
- Check Eureka dashboard: 7 services registered.
- docker images: each ≤300MB.

4 Unit Test Verification

5 min

- Run mvn test from host (not inside container).
- All tests must pass.
- If any test uses hardcoded localhost URLs: update to @Value injection.

PROJECT PROGRESS

Containerized Platform — All Phase 1 Services in Docker

Capability Added

Containerized Platform

docker compose up -d now starts the entire platform reproducibly on any machine with Docker. No more ‘works on my machine’.

Services Containerized

- config-server (8888) · eureka-server (8761)
- api-gateway (8080) · product-service (8081)
- order-service (8082) · payment-service (8083)
- inventory-service (8084)

Enterprise E-Commerce Platform — After Session 9

Docker	Testing I	Testing II	Saga Orch	CI/CD	GitOps	K8s Core	K8s Adv

HOMEWORK · BEFORE SESSION 10

- docker compose down && docker compose up -d — verify data persists (PostgreSQL named volume)
- Pre-reading: @WebMvcTest, @DataJpaTest, TestContainers guide (links in Session Pack)

DAILY QUIZ

8 Questions · 5 Minutes · Google Forms

Multi-Stage · Layer Cache · service_healthy · localhost vs service name · Non-root USER · Image size · Volumes

Mid-Course Exam begins immediately after this quiz — submit Forms before 14:25

MID-COURSE EXAM

Phase 1 Assessment — Exam Starts Now

Duration

120 minutes
14:25 – 16:25
(approx. 3:00–5:30 PM)

Format

Part A: 24 MCQ
(Sessions 1–8, Analyze/Apply)
Part B: Practical coding task

Tools Allowed

Course notes ☑
Spring docs ☑
IDE open ☑
Internet ☒
AI tools ☒

SUBMISSION

git commit -m "mid-course-exam" · git push · open Eureka on your screen for examiner